遙測與空間資訊之分析與應用 - Week 4

國家災害防救科技中心 資訊組
台灣大學土木工程學系 教授
蘇文瑞
wraysu@ntu.edu.tw

The Captain’s Intuition: Where vs. How Much

- **Week 3 Review: We mapped "Where" things are (shelters near rivers).**
- **Week 4 Mission: We map "How intense" the hazard is (terrain).**
- **The Bridge: We don't replace Vector; we use Raster to give Vector "Intelligence."**
- Same shelter, 500m from river — but one is on a hilltop, one in a valley.
- Position is half the answer. Environmental intensity is the other half.
- → Notebook Cell [1]: Reload Week 3 data (townships, shelters)

Week 3 Quick Recap

- **GeoDataFrame = Attributes + Geometry + CRS**
- Three Superpowers: Reprojection, Geometric Properties, Buffer
- sjoin = The spatial VLOOKUP — join by location, not by column value
- .explore() = Interactive Leaflet maps inside Jupyter
- All of these tools carry forward into Week 4.
- New weapon unlocked: Raster data (continuous surfaces)

Advanced Vector: Spatial Aggregation

- **Before Raster, let's master the "Summary" logic.**
- **Dissolve: Merging 368 townships into ~22 counties**
- townships.dissolve(by='COUNTYNAME') → geometries merge too!
- **Groupby + Aggregate: "How many shelters per county? Total capacity?"**
- shelters.groupby('TOWNNAME').agg(count, sum)
- → Notebook Cell [2]: Dissolve + Groupby demo

Lab 1: Vector Aggregation (20 min)

- **Goal: Practice dissolve & groupby before entering the raster world.**
  - Step 1: Dissolve townships → counties (geometry merges)
  - Step 2: Create synthetic shelter data for Hualien County
  - Step 3: Groupby — count shelters & sum capacity per town
  - Step 4: Merge stats back to geometry → .explore() choropleth map
- **Environment: Local Jupyter (same as Week 3)**

The Raster "Glass Box": It’ s Just a Matrix

- **Student Doubt: "Is a .tif just a photo?"**
- **The Truth: A Raster is a NumPy Array with a GPS.**
- Each pixel has one numeric value: elevation, rainfall, temperature...
- In Jupyter, we don't just see colors — we see the raw numbers.
- print(dem.values[0, :5, :5]) → See the actual elevation matrix!
- → Notebook Cell [3]: Create synthetic DEM + inspect the matrix

Why Raster Matters in Disaster Analysis

- **Continuous hazards don't have boundaries like a city polygon.**
- **Slope: Where is it steep? → Landslide potential**
- Aspect: Which direction does water flow? → Hydrology
- Elevation: Where are the low-lying areas? → Flood potential
- These are all "derived products" from a DEM (Digital Elevation Model).
- Week 3: "Which shelters are near rivers?" → Week 4: "How steep is it there?"

Toolstack: RioXarray (The GeoPandas of Raster)

- **rioxarray = xarray + rasterio (spatial extension for arrays)**
- .rio.crs → Check coordinate system (like .crs in GeoPandas)
- .rio.reproject() → Change CRS (like .to_crs())
- .rio.clip() → Cut raster to area of interest
- .rio.resolution() → Pixel size in map units
- You don't need to memorize the API — AI writes it, you verify the output.

Anatomy of a Grid: Pixels & Resolution

- **MOI 20m DEM: Each pixel covers 20m × 20m = 400 m²**
- NoData: "Holes" in the data (ocean, missing areas) — usually -9999 or NaN
- **Resolution Trap: Don't use 1km rainfall data to analyze a 10m bridge!**
- Mismatched resolution = unreliable conclusions
- Taiwan 20m DEM: ~millions of pixels for the whole island
- → Notebook Cell [4]: Examine the Affine Transform metadata

Affine Transform: The Coordinate Translator

- **How does Matrix[5, 10] become real-world (X, Y)?**
- Six parameters: pixel_width, rotation, origin_X, rotation, pixel_height, origin_Y
- real_x = a × col + b × row + c
- real_y = d × col + e × row + f
- **Captain's Check: Is pixel size ~20? Is origin in Taiwan? (not 0.0002 degrees!)**
- → Notebook Cell [4]: Manual verification of one pixel coordinate

CRS Alignment: The #1 Source of Zonal Errors

- **Golden Rule: Vector and Raster MUST share the same CRS.**
- **Strategy: Always reproject Vector to match Raster (not the reverse!).**
- Why? Reprojecting a vector (hundreds of points) is fast.
- Reprojecting a raster (millions of pixels) is very slow.
- Most common error: Shelters in WGS84 (degrees), DEM in EPSG:3826 (meters)
- → zonal_stats returns all NaN → 80% chance it's a CRS mismatch

Infrastructure: Local vs. Cloud (Colab)

- **Local (Windsurf/Jupyter): Best for Git, .env security, offline work**
- **Cloud (Colab): Best for heavy 20m DEM processing (free 12-25GB RAM)**
- **Captain's Strategy:**
- Develop logic locally → Compute on Colab → Save results to GitHub
- Like designing blueprints in the office, manufacturing in the factory.
- Lab 2 will use Colab — your first cloud computing experience!

Getting Started with Google Colab

- **Three essential steps:**
- 1. !pip install rioxarray rasterstats geopandas
- 2. from google.colab import drive; drive.mount('/content/drive')
- 3. Path: /content/drive/MyDrive/GIS_data/dem_20m_hualien.tif
- **Gotchas:**
- Idle 30-90 min → disconnect (variables lost!) | No .env for secrets
- **NEW: VS Code now supports Colab kernels!**
- Install "Google Colab" extension in Windsurf/VS Code → run .ipynb with cloud GPU

Clipping: Focusing Your "Radar"

- **Why process all of Taiwan if the analysis is only for Hualien?**
- dem_clipped = dem.rio.clip(county_boundary.geometry, county_crs)
- Before clip: ~500MB → After clip: ~20MB (save 96% memory!)
- **CRS must match between clip geometry and raster.**
- This prevents Colab from crashing on large files.
- → Notebook Cell [6]: Clip DEM to a sub-region

Computing Terrain: Slope & Aspect

- **Slope = how steep the terrain is (0° flat, 90° cliff)**
- dy, dx = np.gradient(elevation, 20) # 20m pixel spacing
- slope = np.degrees(np.arctan(np.sqrt(dx^2 + dy^2)))
- **Slope > 30° = high landslide potential**
- Aspect = which direction the slope faces (0-360°)
- → Notebook Cell [7]: Compute slope, visualize elevation vs. slope side-by-side

Zonal Statistics: The Ultimate Marriage

- **The Goal: Extract Raster values into Vector shapes.**
- Input 1: Week 3 Shelter Buffers (Vector polygons)
- Input 2: MOI 20m DEM + computed Slope (Raster grids)
- **Output: "What is the average elevation / max slope of each shelter?"**
- rasterstats.zonal_stats(buffers, dem, stats=['mean','max'])
- → Notebook Cell [8]: Zonal stats — shelter buffers × DEM

Lab 2: Raster Exploration on Colab (15 min)

- **Goal: Load real DEM, compute slope, run zonal statistics on Colab.**
- Step 1: Colab setup (pip install + Drive mount)
- Step 2: Load pre-clipped Hualien DEM → inspect shape/CRS/transform
- Step 3: Visualize DEM → Where are the peaks and valleys?
- Step 4: Compute slope → Where is it steepest?
- Step 5: Zonal stats (shelter buffers × DEM) → Terrain risk per shelter
- Step 6: Risk classification (HIGH/MEDIUM/LOW based on slope)

Map Algebra: Pixels Don’ t Lie

- **Formula: NewValue = Function(OldValue)**
- risk_mask = (slope > 30) & (elevation < 100)
- flood_depth = water_level - elevation
- This is the GIS version of Excel formulas — but applied to millions of pixels.
- **AI writes great map algebra, but verify the logic!**
- "Show me all pixels where it's steep AND low" → Boolean risk map

Reclassification: Simplification for Decision

- **Raw slope (0-90°) is too detailed for decision-makers.**
- Reclassify into actionable levels:
- 0-15° → Low Risk (1)
- 15-30° → Medium Risk (2)
- > 30° → High Risk (3)
- **Decision-makers don't read 27.3° — they read "HIGH RISK".**

Infrastructure: Large Files & Security

- **MOI DEM Warning: Full Taiwan .tif > 500MB — HUGE file!**
- Rule 1: Never push .tif to GitHub → add to .gitignore
- Rule 2: Store large files on Google Drive or NAS
- Rule 3: Always clip before analysis (rio.clip)
- **Memory Strategy: clip first, then analyze the small piece.**
- If needed, use chunks parameter for lazy loading.

Professional Visualization: Meaningful Colors

- **Color matters: 'Reds' for risk, 'Blues' for water, 'terrain' for DEM**
- Always add a colorbar with units (Elevation in meters, Slope in degrees)
- Hillshade: Use LightSource for 3D lighting effect on DEM
- Overlay shelter points on DEM: red=high risk, green=safe
- **A good map tells a story without needing explanation.**
- Bad map: random colors, no legend. Good map: intuitive, labeled, clear.

AI Pitfall: Memory Overload

- **AI might try to load the entire Taiwan DEM into RAM → crash!**
- **Captain's Checklist:**
  - "Did you clip the DEM first?"
  - "Are you using chunks for lazy loading?"
  - "Do you need the whole island or just one county?"
- Another trap: AI may reproject the raster (slow) instead of the vector (fast)

Case Study: ARIA v2.0 — Terrain Intelligence

- **Complete workflow:**
  1. Vector: Week 3 shelters + rivers → buffers + sjoin (ARIA v1.0)
  2. Raster: MOI 20m DEM → rio.clip to target county
  3. Derive: np.gradient → slope map
  4. Fuse: zonal_stats → mean_elevation + max_slope per shelter
  5. Judge: Composite risk = river distance + slope + elevation
  6. Output: terrain_risk_audit.json + terrain_risk_map.png

Week 4 Reflection

- **"Vector is the bone; Raster is the muscle."**
  - We moved from "Where" (Week 3) to "How Much" (Week 4).
  - Zonal statistics = the ultimate marriage of vector and raster.
- **Discussion:**
  - "If you only have vector, what do you miss?" → Intensity
  - "If you only have raster, what do you miss?" → Location names, attributes

Homework: ARIA v2.0 + Next Week Preview

- **Assignment: ARIA v2.0 — Terrain Intelligence Upgrade**
- Extend Week 3 river analysis with DEM slope + elevation risk
- Use Colab for DEM processing → results back to GitHub
- Deliverables: ARIA_v2.ipynb, terrain_risk_audit.json, terrain_risk_map.png, README
- **Next Week: Week 5 — Real-time APIs & Dynamic Monitoring**
- Weather APIs, real-time rainfall, from static analysis to dynamic alerts